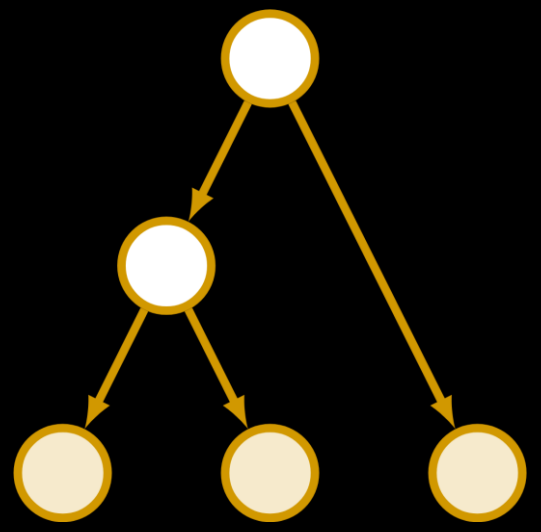


BIOL 531: RevBayes Intro

First session: Setup, Intro, Basics

Orlando Schwery, 09. Mar. 2026, University of Kentucky

Quick Intro

Orlando Schwery

- BSc & MSc U of Zürich, Switzerland
- PhD U of Tennessee
- Postdoc New Mexico Consortium/U of Idaho [in Harmon Lab!]
- Postdoc Southeastern Louisiana U/LSU
- Postdoc/Research Scientist: Virginia Tech [with Dr. Corn!]
- Research Scientist: U of Kentucky

- Phylogenetics, Diversification, Biogeography
- Model Adequacy, Causal Inference
- Ericaceae, Dung Beetles, …?

The Actual People Behind This

Höhna, Landis, Heath, Boussau, Lartillot, Moore, Huelsenbeck, Ronquist. **2016**. RevBayes: Bayesian phylogenetic inference using graphical models and an interactive model-specification language. *Systematic Biology*, 65:726-736.

Höhna, Heath, Boussau, Landis, Ronquist, Huelsenbeck. **2014**. Probabilistic graphical model representation in phylogenetics. *Systematic Biology* 63:753–771.

Sebastian Höhna

Tracy A. Heath

Bastien Boussau

Michael J. Landis

Fredrik Ronquist

John P. Huelsenbeck

Nicolas Lartillot

June Walker

Ben Redelings

Joëlle Barido-Sottani

Brian R. Moore

The Plan…

1. Introduction: The why and how of RevBayes
   (Basics of Rev Language and comparison to R)

2. Application: Setting up a simple model
   (Simple Brownian Motion)

3. Mix & Match: The strength of RevBayes
   (Joint inference)

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

Can’t we do all that in R…?

Setting up analyses in RevBayes can seem like a lot…

Code for BiSSE in R (diversitree)

```r
library (diversitree)

# Prep Data
phy<-read.tree("Terapontidae.tre")
X <- read.csv("SSE.csv", row.names = 1)
Diet <- setNames(X$Diet,rownames(X))

# run ML
samplingf<-c(0.58, 0.78, 0.85)
p <- starting.point.musse(phy, k=3)
lik.musse<-make.musse(phy,Diet,3,sampling.f=samplingf)
fit.full<-find.mle(lik.musse,x.init=p[argnames(lik.musse)])
summary(fit.full)

# run MCMC
prior <- make.prior.exponential(1/2)
prelim <- mcmc(lik.lambda.mu, fit.lambda.mu$par, nsteps=100, prior=prior,
    w=1, print.every=0)
w <- diff(sapply(prelim[2:(ncol(prelim)-1)], quantile, c(0.05, 0.95)))
mcmc.lambda.mu <- mcmc(lik.lambda.mu, p[colnames(w)], nsteps=1000, prior=prior, w=w, print.
every=100)

# plot results
colors<-setNames(c("yellow","red","green"),1:3)
profiles.plot(mcmc.lambda.mu[,grep("lambda",colnames(mcmc.lambda.mu))], col.line=colors,
las=1, legend.pos="topright")
```

Can’t we do all that in R…?

Setting up analyses in RevBayes can seem like a lot…

Code for BiSSE in R (diversitree)

```r
library (diversitree)

# Prep Data
phy<-read.tree("Terapontidae.tre")
X <- read.csv("SSE.csv", row.names = 1)
Diet <- setNames(X$Diet,rownames(X))

# run ML
samplingf<-c(0.58, 0.78, 0.85)
p <- starting.point.musse(phy, k=3)
lik.musse<-make.musse(phy,Diet,3,sampling.f=samplingf)
fit.full<-find.mle(lik.musse,x.init=p[argnames(lik.musse)])
summary(fit.full)

# run MCMC
prior <- make.prior.exponential(1/2)
prelim <- mcmc(lik.lambda.mu, fit.lambda.mu$par, nsteps=100, prior=prior,
    w=1, print.every=0)
w <- diff(sapply(prelim[2:(ncol(prelim)-1)], quantile, c(0.05, 0.95)))
mcmc.lambda.mu <- mcmc(lik.lambda.mu, p[colnames(w)], nsteps=1000, prior=prior, w=w, print.
every=100)

# plot results
colors<-setNames(c("yellow","red","green"),1:3)
profiles.plot(mcmc.lambda.mu[,grep("lambda",colnames(mcmc.lambda.mu))], col.line=colors,
las=1, legend.pos="topright")
```

Code for BiSSE in RevBayes

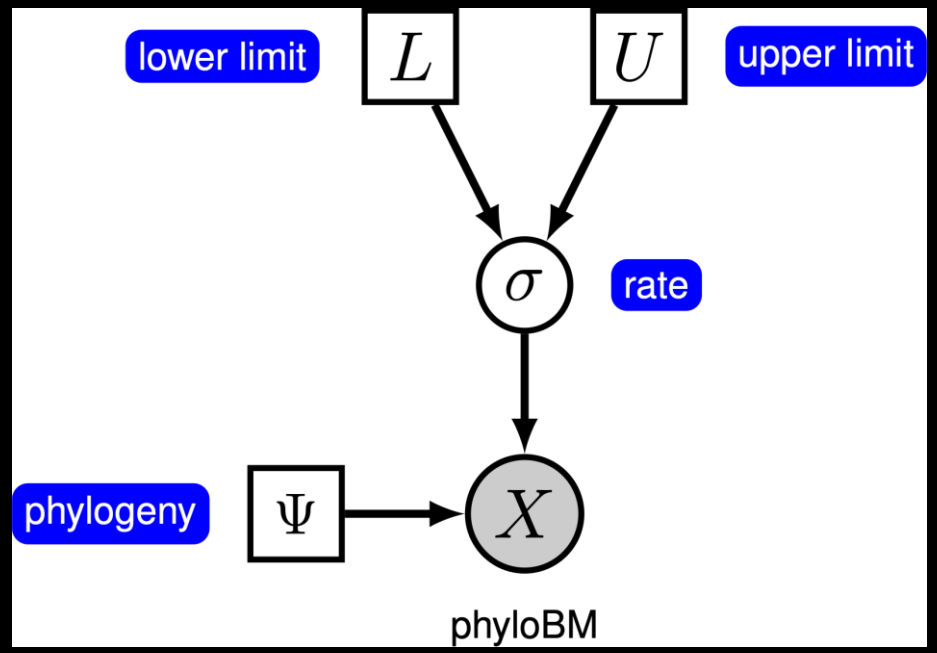
```
# BiSSE Model
# Orlando Schwery, 8. Feb. 2023

# priors on div rates

mean_lam := ln(extant_taxa/2)/tree.rootAge()
mean_mu := mean_lam/10
rate_lam <- 1/mean_lam
rate_mu <- 1/mean_mu

for (i in 1:NUM_STATES) {


    # speciation rate
    speciation[i] ~ dnExponential(rate_lam)
    speciation[i].setValue(mean_lam)
    moves.append(mvScale(speciation[i], lambda=0.20, weight=3, tune=TRUE))

    # extinction rate
    extinction[i] ~ dnExponential(rate_mu)
    extinction[i].setValue(mean_mu)
    moves.append(mvScale(extinction[i], lambda=0.20, weight=3, tune=TRUE))

}



# transformations for convenient comparison
for (i in 1:NUM_STATES) {
    netDiv[i] := speciation[i] - extinction[i]
    turnover[i] := speciation[i] + extinction[i]
    extinction_frac[i] := extinction[i] / speciation[i]
}


# priors on transition rate

# the rate parameter stays the same
rate_pr := tree.treeLength() / 10
# except the lower one for the unlikely reversal of viviparity
# rate_pr_dollo := tree.treeLength() / 2


# transition rates
for (i in 1:NUM_STATES) {
  transition_rates[i] ~ dnExponential(rate_pr)
  transition_rates[i].setValue(1/rate_pr)
  moves.append(mvScale(transition_rates[i], lambda=0.20, tune=TRUE, weight=3.0))
}

# turn this into a rate matrix again
Q := fnFreeK( transition_rates, rescaled=FALSE)

# we estimate the root frequencies as a free parameter

root_state_freq ~ dnDirichlet( rep(1, NUM_STATES) )
moves.append(mvBetaSimplex(root_state_freq, alpha=0.5, weight=2))
moves.append(mvElementSwapSimplex(root_state_freq, weight=2))

obs_taxa <- tree.ntips()

sampling <- obs_taxa / extant_taxa

root_age <- tree.rootAge()

sse ~ dnCDBDP(rootAge          = root_age,
              lambda           = speciation,
              mu               = extinction,
              Q                = Q,
              pi               = root_state_freq,
              rho              = sampling,
              condition        = condition,
              simulateCondition = simcondition,
              minNumLineages   = 2,
              maxNumLineages   = 5000,
              exactNumLineages = obs_taxa,
              maxTime          = 1000)  # default

sse.clamp(tree)
sse.clampCharData(tip_data)
mymodel = model(sse)
```

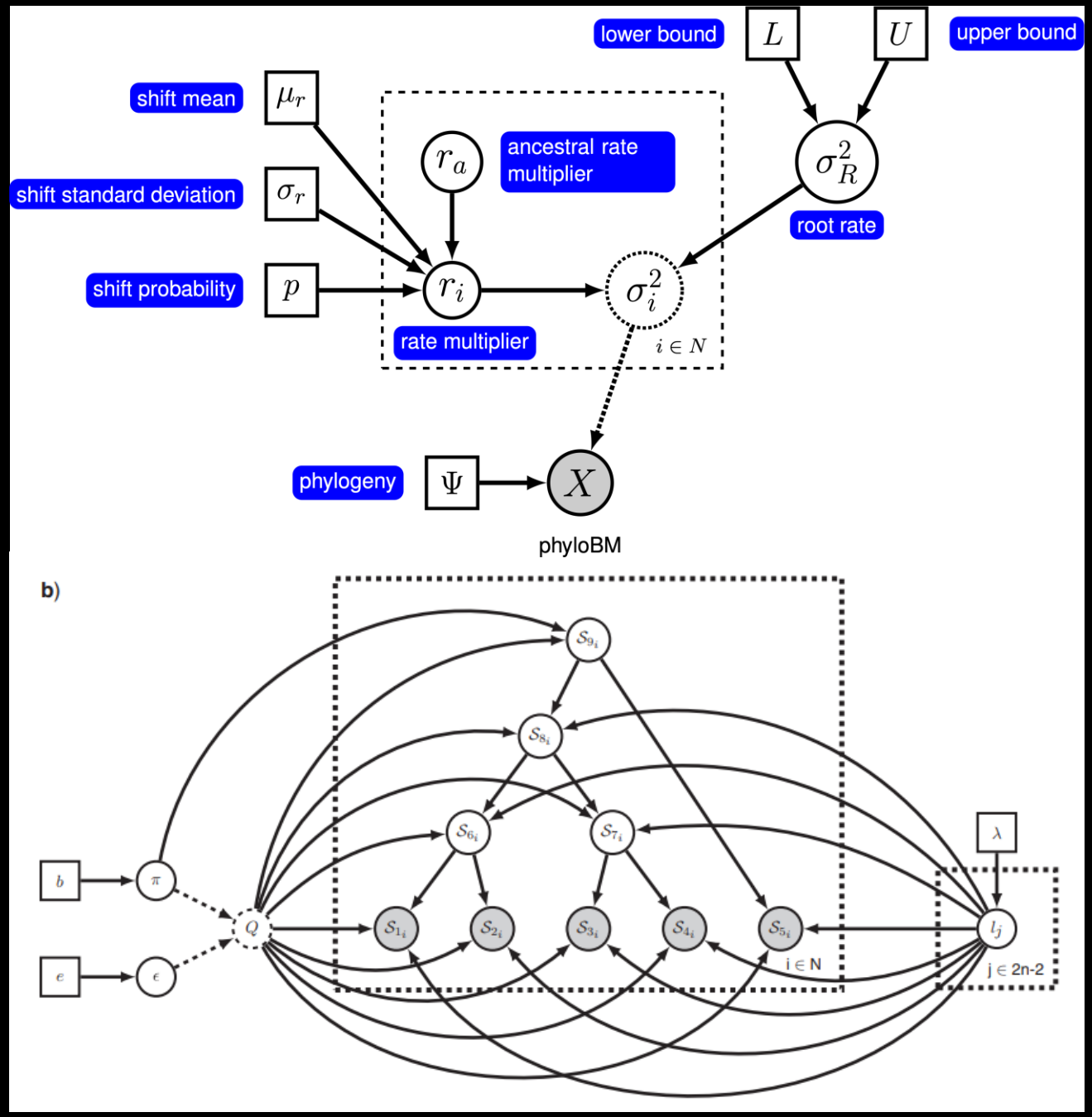
Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility

DAGs





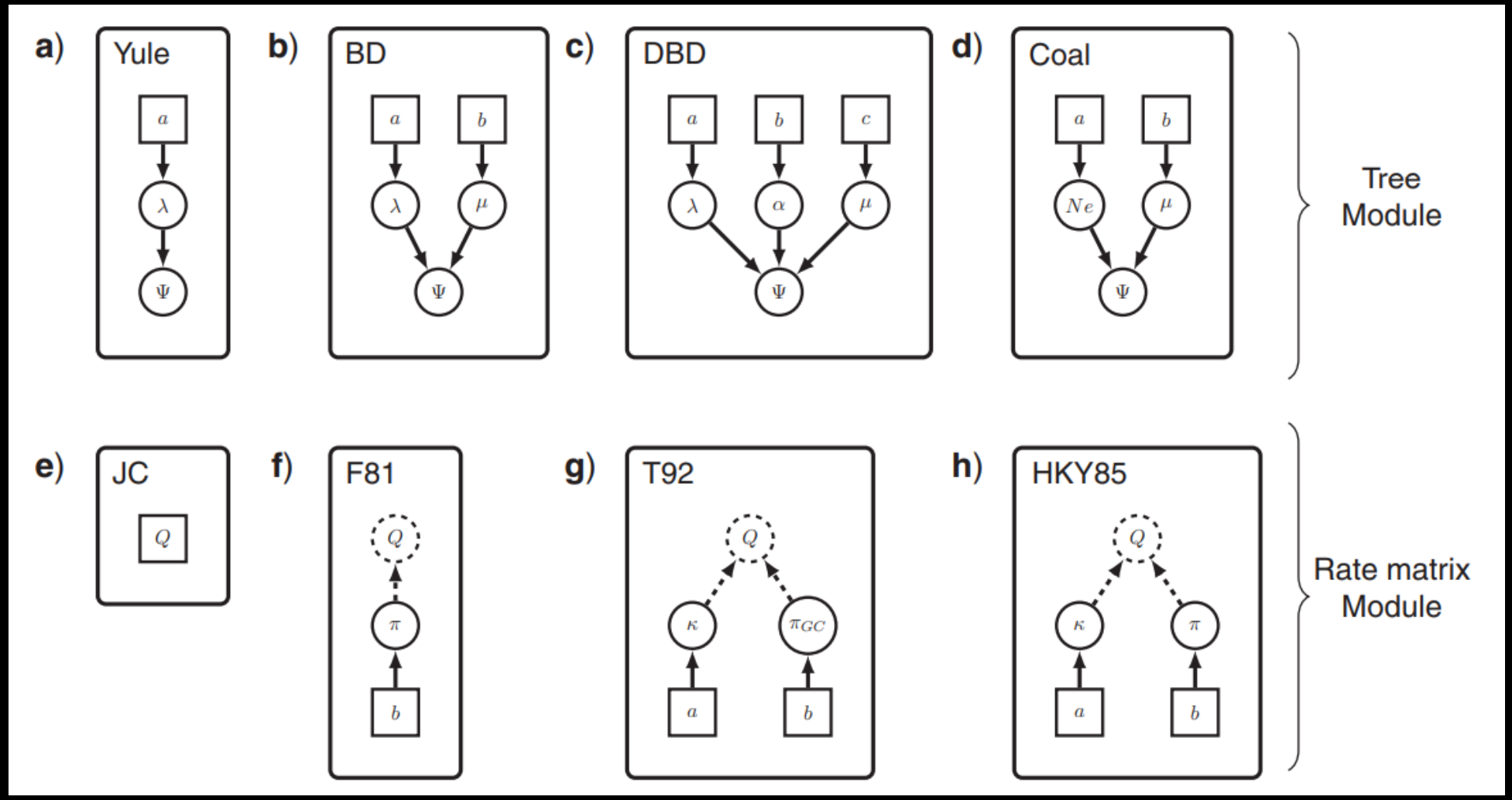
Höhna *et al.* 2014 Syst. Biol.

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility
  - Modular: decomposing and reassembling models

Modular Structure



Höhna *et al.* 2014 Syst. Biol.

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility
  - Modular: decomposing and reassembling models
  - Flexible: customizable model and inference machinery

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility
  - Modular: decomposing and reassembling models
  - Flexible: customizable model and inference machinery
  - Generative: Same graph for inference and simulation (testing, adequacy, …)

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

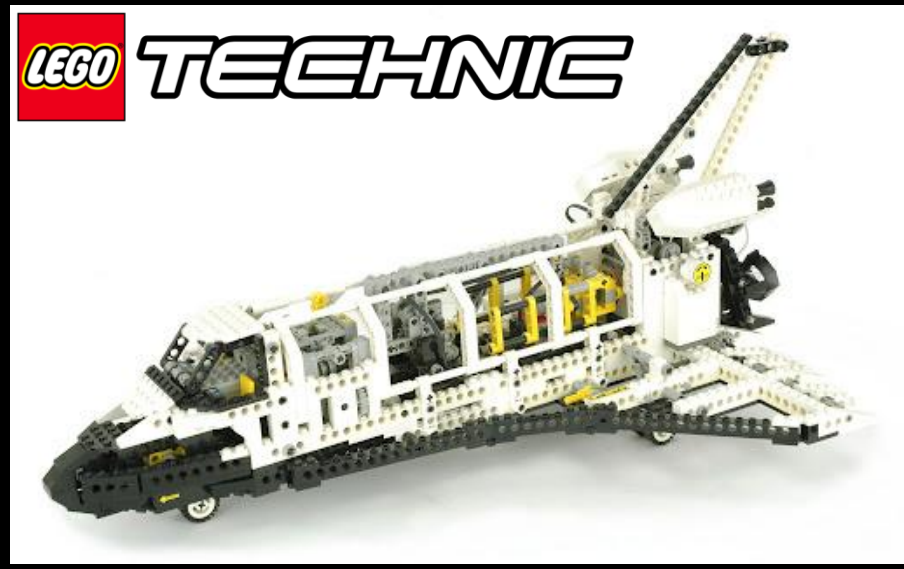
  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility
  - Modular: decomposing and reassembling models
  - Flexible: customizable model and inference machinery
  - Generative: Same graph for inference and simulation (testing, adequacy, …)
  - Fast: *Rev* front-end and *C++* back-end

Can’t we do all that in R…?

- Initial reaction:
  “Why not just use the well-established phylo-universe in the R-framework?”

  - DAGs: Explicit representation of assumptions and variable dependencies
    → Transparency and reproducibility
  - Modular: decomposing and reassembling models
  - Flexible: customizable model and inference machinery
  - Generative: Same graph for inference and simulation (testing, adequacy, …)
  - Fast: *Rev* front-end and *C++* back-end
  - Integrated : expandability of modeling due to unified architecture

Benefits vs Learning-Curve



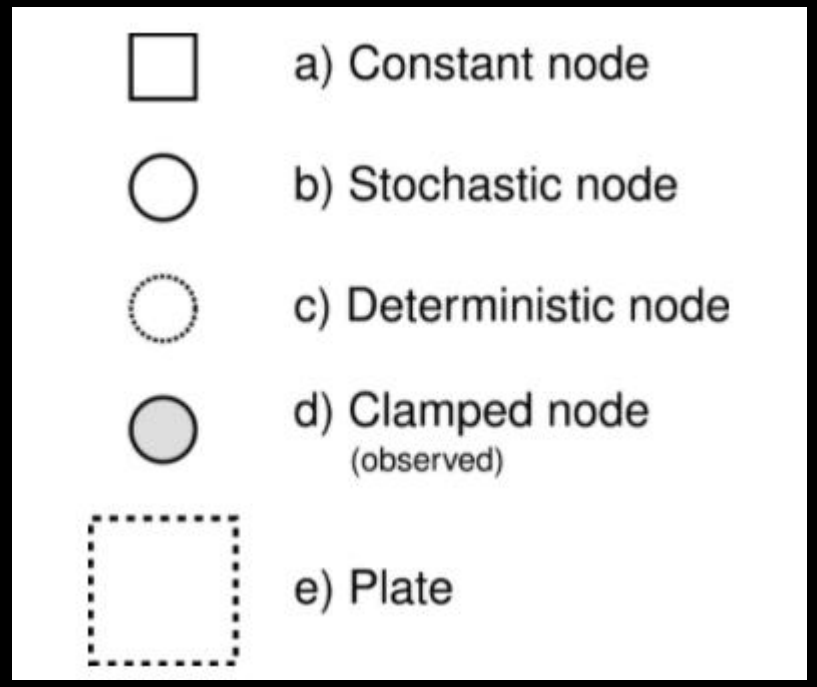
Starting RevBayes & Basic Operations

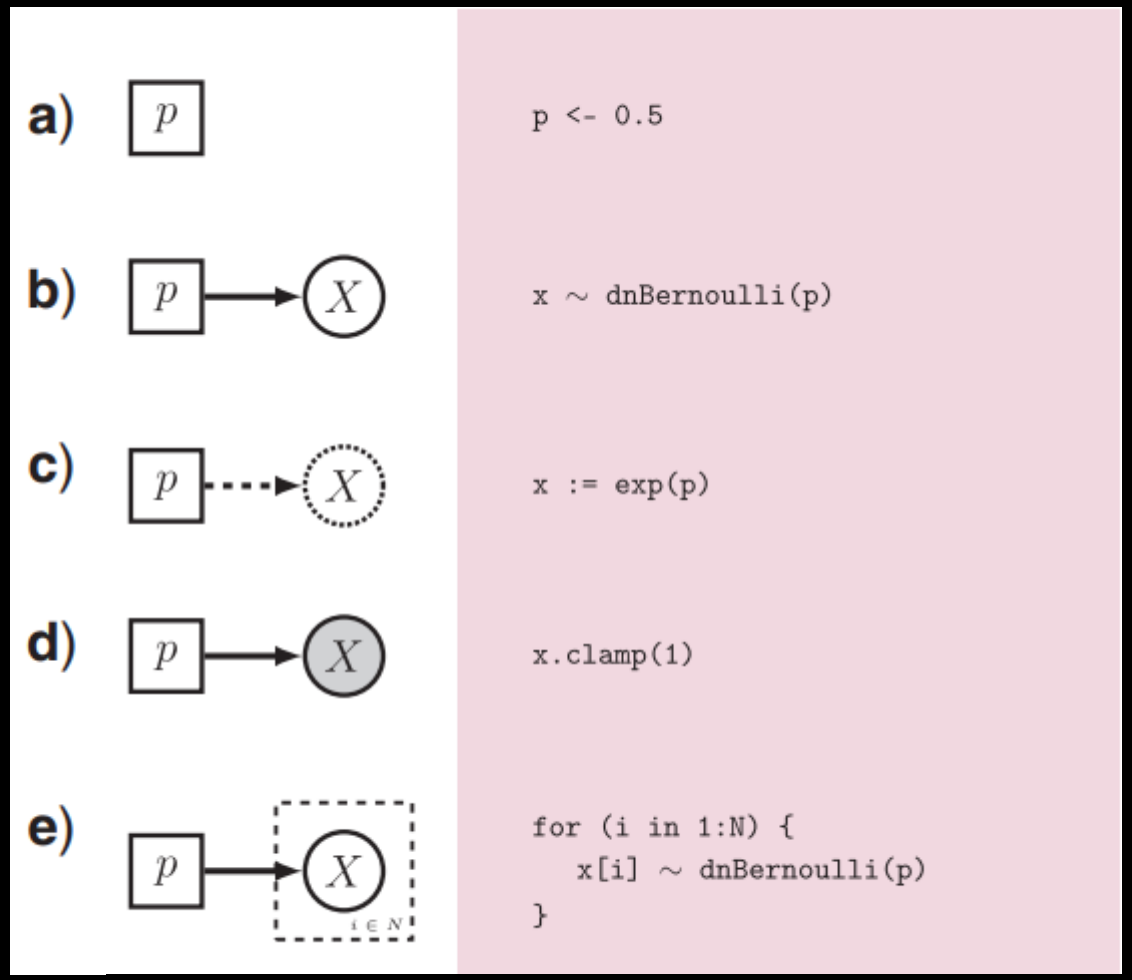
- Use from command-line
- Interactive or script-running
- R package RevGadgets for post-processing and plotting
- Possibility to set up RStudio, VSCode or Jupyter as GUI, R package Revticulate as interface…

Syntax and more…

[code-time]

Graphical Models



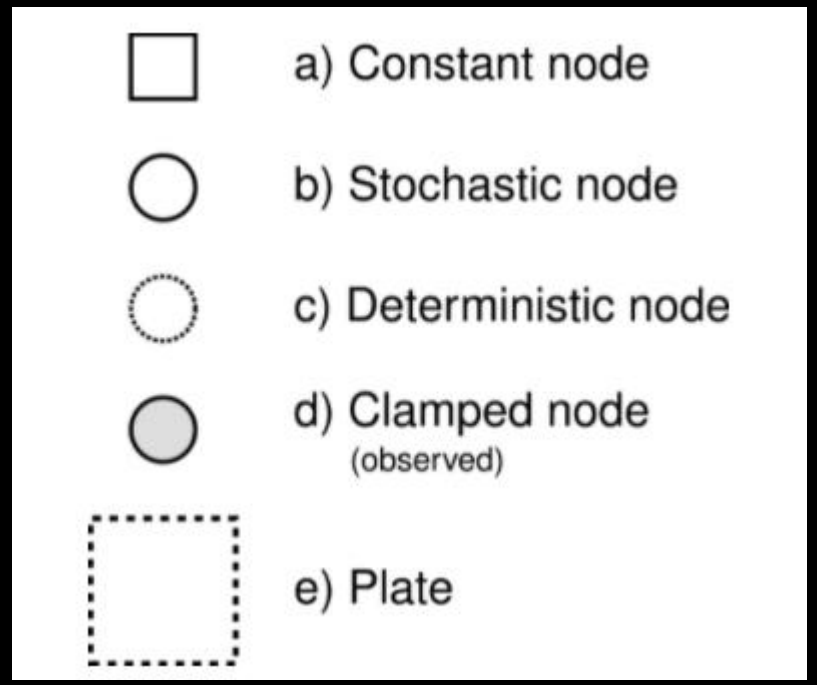


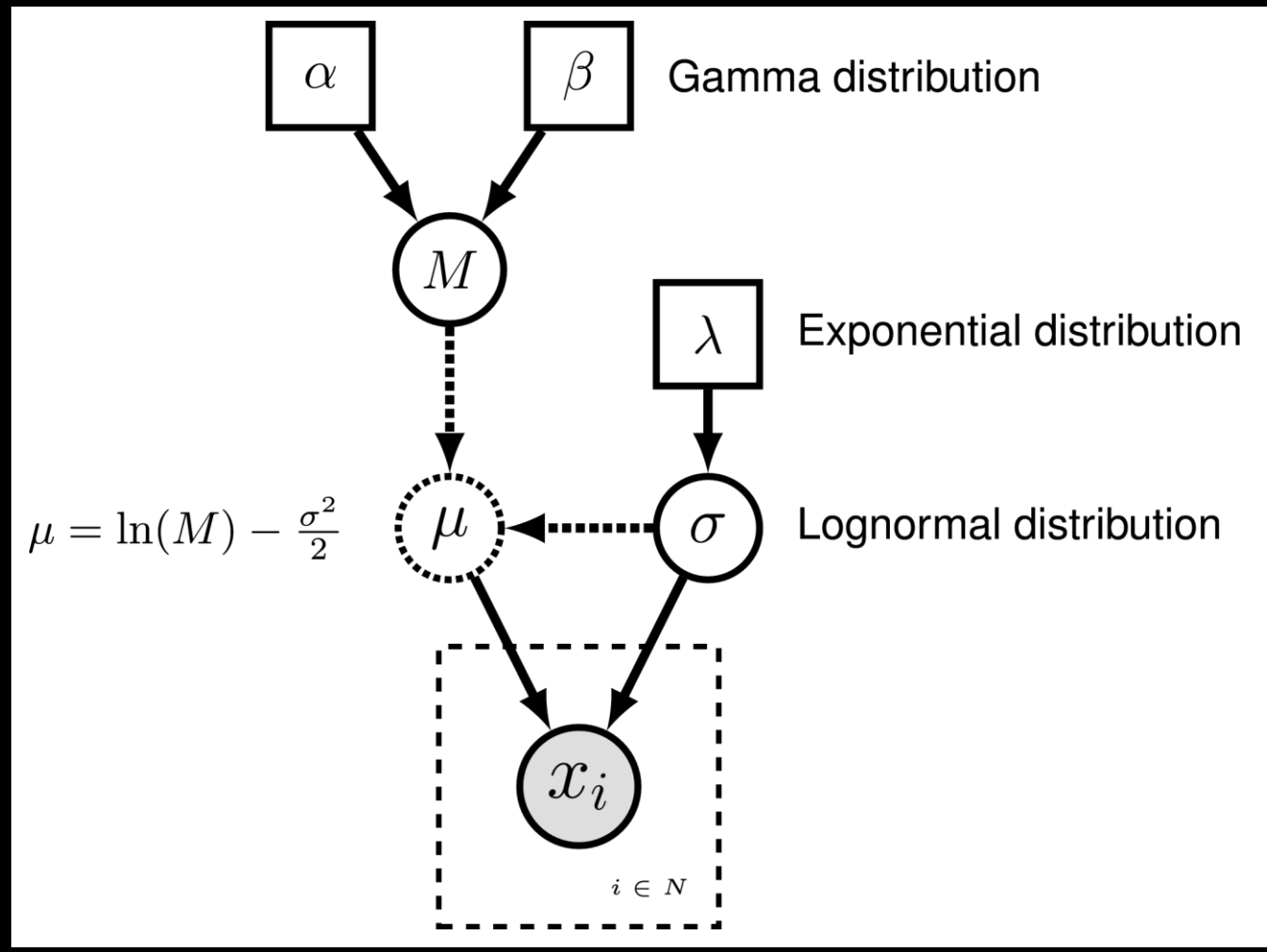
Directed Acyclic Graph (DAG)

→ Nodes (vertices) and Edges (circles/squares and arrows)

Höhna *et al.* 2016 Syst. Biol.

Graphical Models





Lognormal Model/Distribution: $x = e^{\mu+\sigma z}$

[μ: location parameter (log mean); σ: standard deviation]